

SD TCL - Processes and Phantoms

Seeing what a session is doing, running work in the background, and taking a snapshot of a running program.

Every SD session is a process with a **user number**, and almost everything on this page takes one. The number is not the Windows process id and it is not the account name: it is SD's own handle on a session, it is reused after a session ends, and it is what `pstat` and `pdump` both expect.

A **phantom** is a session with no terminal. You start one, it runs a command in the background under its own user number, and what it would have displayed is written to a file instead.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The listings on this page were produced by running them, on SD Core for Windows W1.0-0. The two exceptions are `phantom` and `pdebug`, and the reason is on the page: neither can be driven down a pipe, so both are described from their source rather than shown.

What is not on this page

Seeing other people's sessions, and ending them, are administrator verbs. `listu` lists every session on the machine and `logout n` ends one, and neither is in an ordinary account's VOC. They are documented in the **administrator documentation**, under *Sessions and Locks*, which is a separate set your administrator may or may not have given you.

`logout` **with no argument is the exception and every account has it** — it ends your own session and is `quit` under another name. That is worth knowing before typing it intending to list something.

What is here is **your own processes**: what this session is doing, work you started in the background, and how to look at either.

What a session is doing: `pstat`

```
pstat {user n} {level n} {no.page} {lptr {n}}
```

With no arguments it reports every session; `user n` restricts it to one.

```
:pstat
User Detail
  27 Account: DON
      Command: pstat
      $PSTAT 141 (262)
  12 (Not responding)
```

Each line under a user number is the account it is in, the command it is running, and where in that command it currently is — program name, source line, and the address in hexadecimal.

THE LEVELS ARE ADDITIVE

default	the current program only
---------	--------------------------

level 1	the whole call stack
----------------	-----------------------------

level 2	the internal subroutine stack — where each <code>gosub</code> will return to
----------------	---

level 3	both
----------------	------

From inside a running program, which is where the levels are worth having — at the prompt there is nothing on the stack but `pstat` itself:

```
User Detail
  27 Account: DON
      Command: pstat user 27 level 1
      $PSTAT 141 (262)
      Command processor
      /cygdrive/c/ProgramData/SD/user_accounts/don/BP.OUT/ZZMATH 34 (152)
      Command processor
```

Read it downwards: `$PSTAT` is running, called by the command processor, which was reached by `execute` from the program `ZZMATH` at line 34, which the command processor started. **level 3 adds an indented line under each frame** for the `gosub` inside it.

(NOT RESPONDING) MEANS THE PROCESS IS GONE, NOT BUSY

`pstat` does not read another session's memory. It **asks** — it posts an event to the target and waits for the answer, for up to four seconds. A session that never answers gets that line.

That is the cheapest way to find a dead session. A session killed from outside SD keeps its entry in the user table and its slot is still counted; `pstat` is what tells you nothing is behind the entry. User 12 in both listings above is one of those. **Clearing it is an administrator's job** — reporting the user number is the useful thing you can do.

Running something in the background: phantom

```
phantom command
```

The command is everything after the verb. SD creates a new process, gives it a user number, and answers at once:

```
Started phantom user 31
```

Your session does not wait, and the phantom has no terminal, so anything it would have displayed goes to a **como** record instead — in the account's `$COMO` file, named `PH n _ ddmm yy _ hhmm ss` for the phantom's user number and start time. `$COMO` is created if the account has not got one. The first line in the record is the phantom's own banner:

```
Phantom 31 started at 14:22:07 27 Aug 2026
```

so a phantom that produced nothing else is still distinguishable from one that never started.

REFUSAL	WHEN
<i>Phantom command missing</i>	<code>phantom</code> with nothing after it
<i>Phantom processes cannot be started within a transaction</i>	inside <code>begin transaction</code>
<i>Failed to create phantom process</i>	the operating system would not create the process

A phantom cannot be started from a scripted session, and this is the one to know. When SD is fed commands down a pipe, the phantom child inherits that pipe. The job then never completes — not even after the parent session has exited — and the only way out is to kill the process, which leaves an entry in the user table that needs an elevated `sd -cleanup` to clear. **phantom is for a person at a prompt, or for a program, and not for a piped script.** The listings above are quoted from the verb's own message texts for that reason.

A phantom is not a scheduled job. It runs once, now. Recurring work is a Windows scheduled task that starts SD, and that is a different subject.

WHAT A PHANTOM DOES NOT INHERIT

A phantom is a new SD process, not a branch of yours. It is started with `fork` and `exec` and goes through login of its own, so **nothing your session holds in memory reaches it** — no `@`-variables, no unnamed common, no open files, no active select list. **The command string is the only thing passed**, through a record in SD's `$IPC` file, and the session's option settings go with it only when the `INHERIT` option is set.

What phantoms have I started: `status`

```
status

:status
There are no phantom processes started by this process
```

which is the answer in any session that has not started one. When there are some, it prints a user number, the start time and the command for each:

```
User      Started      Command
```

It reports only your own children. It is not a system-wide view and it is not `listu` — a phantom somebody else started does not appear, and neither does one your session started before you `logto` 'd somewhere else. The register it reads is keyed by the parent's user number.

Dumping a process: `pdump`

```
pdump n
```

Writes a snapshot of session *n* to a file and carries on; nothing is stopped and nothing is lost. The file is `sddump.n` in the directory named by the `DUMPDIR` configuration parameter, or in SD's own system directory when `DUMPDIR` is empty, which is how it ships:

```
Dumping process state as C:\ProgramData\SD\sdsys/sddump.27
```

It holds the `@`-variables, the current sentence and command, the call stack, open files and their locks, and named and unnamed common — which is to say **it holds application data**, and it is written where any SD user can read it. Treat a dump as you would the data of the program that produced it, and delete it when the question it was written to answer has been answered.

REFUSAL	WHEN
<i>User number required</i>	<code>pdump</code> with no number
<i>Not logged in</i>	no session has that user number

PDUMP not allowed for processes run under other usernames

the `PDUMP` configuration parameter has bit 1 set and the session is not elevated

It is an event, not a call. `pdump` marks the target and returns; the target writes the file when it next looks at its event flags. A process that is wedged somewhere that never checks will not produce one.

Debugging a phantom: `pdebug`

<code>pdebug</code>	attach to a phantom that is waiting for a debugger
<code>pdebug command</code>	start command as a phantom and attach to it

`pdebug` puts the SD debugger on **this** terminal and the program in a **phantom**, which is the only way to debug something that has no terminal of its own. With a command it starts the phantom for you; with none it waits for one to ask for a debugger, and `q` gives up waiting.

It refuses inside a phantom or an API session — *Phantom debugger can only be executed from an interactive session* — because it needs a terminal to draw the debugger on.

Do not send `pdebug` down a pipe. It polls the keyboard while it waits, and in a piped session the input stream is the script, so it consumes the commands that have not run yet. Together with the phantom it starts, that is two ways for one verb to hang a scripted session. **It is described here from source for exactly that reason.**

The debugger itself — the commands it takes once it is attached — is in SD Basic - Debugging.

Who has these verbs

standard

`status`

programmer

`phantom` `pstat` `pdebug` `pdump`

The only one every account has is `status`, and it reports nothing but that account's own phantoms. The rest are programmer verbs: making a background process, and looking at one.

There are two gates and they are not the same one. The tier decides whether you have the verb at all; **elevation decides whether it does anything to somebody else.** `pstat` will report any session; `pdump` *n* is yours to use on your own processes and refuses for another Windows account's without an elevated session.

See also

[SD TCL - Locks](#) · [SD TCL - The Terminal and the Session](#) · [SD Basic - Debugging](#).